R workshop part 1 - Introduction

Jan Gačnik

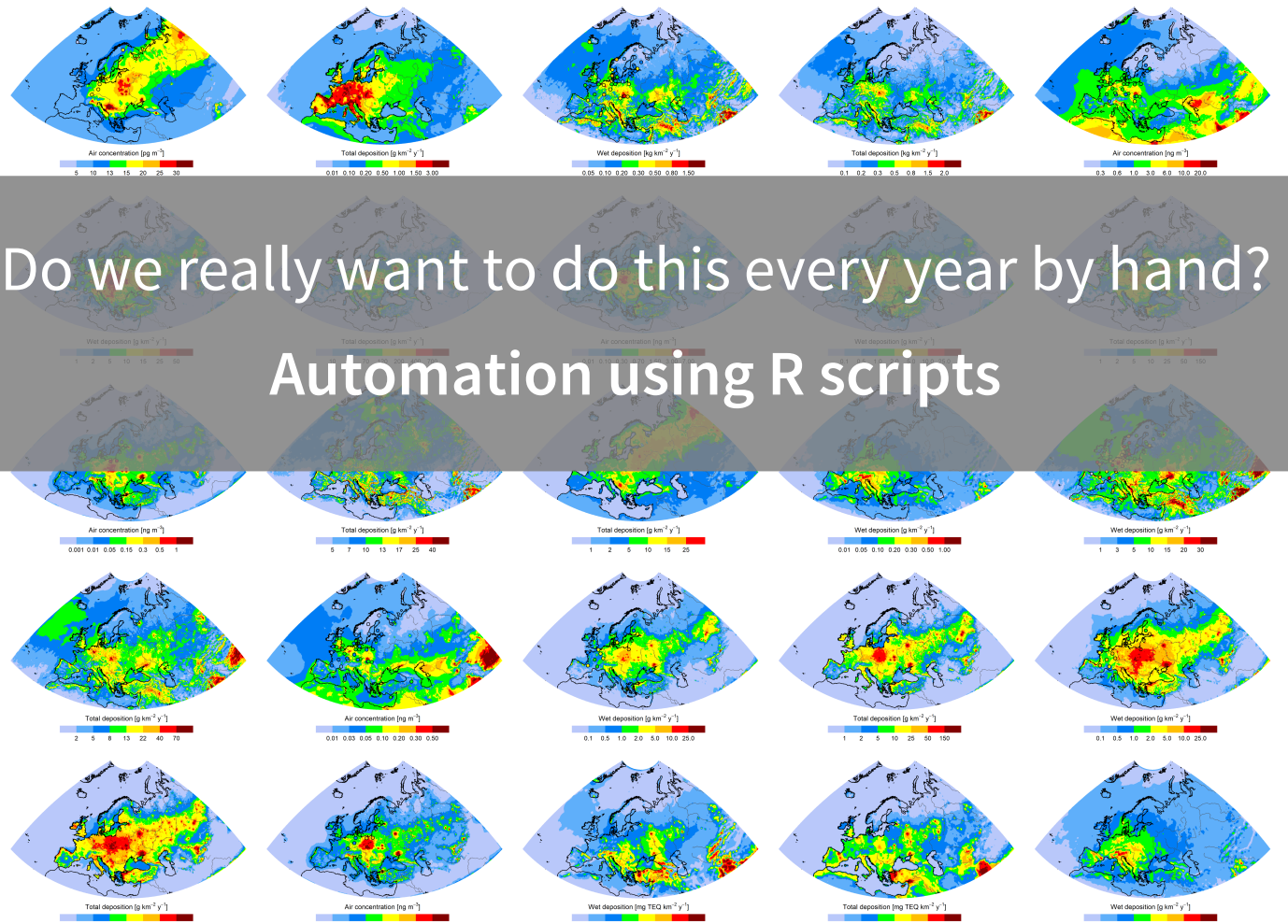
Prepared: 2026-07-30

Why R?

R is a programming language built for statistical computing

If one already knows Excel, Origin, SigmaPlot, etc. - why use R?

- *Free* to use
- *Large* community - support and packages
- Analysis and visualization are *easy to replicate*
- *Freedom* - almost anything is possible and modifiable
- *Automation* of plotting



Do we really want to do this every year by hand?

Automation using R scripts

Why R?

R vs python

Analogy: With what can you cook rice?

Very good rice can be cooked

…but also any other dish too

Perfect rice can be cooked

…but very few other dishes

Why R?

Why not just ChatGPT & Claude ?

- People who know R get dramatically more value from AI assistants than beginners
- AI as a copilot – you still need to know how to fly

Other considerations

- R script is a reproducible record of what was done with data. AI conversation is not.

ARIS and FAIR data

Open access to all research outputs from publicly funded projects - **data analy and visualization included**

Overview

1. R and R studio orientation
2. Creating and using objects
3. Packages
4. Basic visualization
5. Basic analyses
6. Demonstration on a precipitation isotopes dataset

Workshop materials based on:

- Hadley Wickham, R for Data Science, https://r4ds.hadley.nz/
- Charles Lanfear, Introduction to R, https://clanfear.github.io/Intro_R_Workshop/

1) R and R studio orientation

RStudio

RStudio is a user-friendly application that gives a visual interface for writing, editing, and running R code.

RStudio can:

- Write & run R code with helpful features like auto-complete and syntax highlighting
- Enables viewing created objects (tables, graphs)
- Files, code, graphs, outputs → all in one window

Editing and running code

Three ways to run R code in RStudio:

1. **Highlighting the lines in the editor window** and clicking *Run* at the top. Alternatively, `Ctrl` + `Enter` for Windows or ⌘ + `Enter` for Mac

2. **Clicking** anywhere on the code line (in the **editor window**) and pressing `Ctrl` + `Enter` or ⌘ + `Enter`

3. Typing or pasting the code directly into the **console window** and pressing `Enter`

Incomplete code

What if you mess up (e.g., forget to close brackets)?

- In console window, R might show `+` sign, indicating that the command has to be finished

```
> 12 * (2 + 6
+
```

- You can finish the command or hit `Esc` to get out of it

R as a calculator

In the console, type `12 + 34 + 56` and hit `Enter`.

- The outcome should look like this:

```
1  12 + 34 + 56
```

```
[1] 102
```

- The `[1]` indicates the numeric index of the first element in that line

In your editor window, try typing the line `sqrt(400)` and running it by `Ctrl` + `Enter` or ⌘ + `Enter`

```
1  sqrt(400)
```

```
[1] 20
```

Functions and help

`sqrt()` was an example of a **function** in R. To see more details about what each function does, type `?` before the function.

```
1  ?sqrt
```

Arguments are inputs that are passed to a function. In the case of `sqrt()`, the only argument is `x`, which can be a number, or a vector of numbers

Functions can get complex – documentation is your friend!

Comments in the code

By using the `#` sign, you can create a comment, which will be ignored when you run code.

- Comments are useful for describing what was done:

```
1 # This calculates square root of 400
2 sqrt(400)
```

```
[1] 20
```

- Comments are also useful to “comment-out” the code you want to ignore

```
1 # This will be skipped
2 # sqrt(400)
```

2) Creating and using objects

Creating and using objects

R stores *everything* as an object - data, functions, models, output. Objects are created using the assignment operator `<-`, shortcut for it: `Alt` + `-`

- Create an object, display it, and use it in calculations:

```
1 new_object <- 144
2 new_object
```

```
[1] 144
```

```
1 new_object + 10
```

```
[1] 154
```

```
1 sqrt(new_object)
```

```
[1] 12
```

Creating vectors

A **vector** is a series of elements, such as numbers or characters

You can create a vector and store is as an object. Creating vectors is done using the function `c()` which stands for “combine” or “concatenate”:

```
1 numeric_vector <- c(4, 9, 16, 25, 36)
2 numeric_vector
```

```
[1]  4  9 16 25 36
```

- If you name an object the same name as an existing object *it will overwrite it*
- You can provide vectors as arguments to many functions:

```
1 sqrt(numeric_vector)
```

```
[1] 2 3 4 5 6
```

Creating character vectors

We often work with categorical data. A vector of text elements (called **strings** in programming jargon) is created by putting text in quotes:

```
1 string_vector <- c("Atlantic", "Pacific", "Arctic")
2 string_vector
```

```
[1] "Atlantic" "Pacific"  "Arctic"
```

- Functions can also be used on character vectors, such as the `sort()` function

```
1 sort(string_vector)
```

```
[1] "Arctic"   "Atlantic" "Pacific"
```

- As we go on, we will see functions which accept more than just one argument, such as the `decreasing = TRUE` argument of the `sort()` function:

```
1 sort(string_vector, decreasing = TRUE)
```

```
[1] "Pacific"  "Atlantic" "Arctic"
```

More complex objects

Same principles show with vectors can be used with more complex objects, such as **matrices**, **arrays**, **lists**, and **dataframes**

- **Dataframe** is the object we will focus on the most

Easiest way to import data into R – *delimited text files* (e.g., .csv), but many data formats can be imported (.xlsx, .nc, .shp, .json, …)

- For *delimited text files*, `read.table()` and `read.csv()` are common import functions

```
1  new_df <- read.csv("Some_spreasheet.csv")
```

Accessing data in objects

There are two main ways to access data inside objects: square brackets (`[]` or `[[]]`), and `$`. How you access an object depends on its *dimensions*.

Dataframes have *2* dimensions: **rows** and **columns**. Using square brackets we can **subset** it using the format: `object[row, column]`. If you leave row or column place empty, all elements in that dimension will be selected.

```
1 penguins[2,] # Second row
```

```
  species    island bill_len bill_dep flipper_len body_mass    sex year
2  Adelie Torgersen     39.5     17.4         186      3800 female 2007
```

```
1 penguins[1:3, 3:5] # Rows 1-3 and columns 3-5
```

```
  bill_len bill_dep flipper_len
1     39.1     18.7         181
2     39.5     17.4         186
3     40.3     18.0         195
```

3) Packages

Installing and loading packages

Packages contain useful functions (and sometimes data) created by the community

- These are not included in base R
- The Comprehensive R Archive Network (CRAN): packages go through a *peer-review process by statisticians and computer scientists*

CRAN packages are installed using `install.packages()` – only once is needed

```
1 install.packages("tidyverse") # Run first
2 library("tidyverse") # Run second
```

Useful R packages

Too many to list, but `tidyverse` is by far the most useful

A package that includes a *family of packages*:

- `ggplot2` for *plotting*
- `dplyr` and `tidyr` for *data manipulation*
- `lubridate` for working with *dates*
- ...

Other notable packages: `readxl` (reading Excel files), `sf` (spatial data), `tidymodels` (modelling & machine learning), `data.table` (big data handling), and many more.

Loading packages

After installation, packages have to be loaded into the environment using `library()`:

```
1 library(tidyverse)
```

Once a package is loaded, you can use functions and/or data inside them:

```
1 data("diamonds") # Places data in your global environment
2 head(diamonds) # Displays first six elements of an object
```

```
# A tibble: 6 × 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
```